AIMERZ.ai
Aim.Act.Achieve

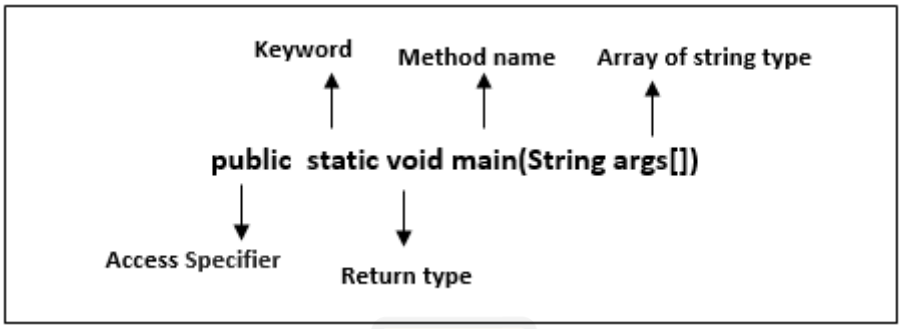
NOTES

Fundamentals of Java Programming -3

(Methods in Java & Control flow of Method's-(Main Method))

Main Method in Java

The main method is the entry point of any Java program. When a Java application starts, the main method is the first method that is executed. Every Java program has at least one main method. It must be defined in this exact format:



Break Down of Main Method

1. **public:** This keyword makes the main method accessible from anywhere. It's required to allow the JVM to access and execute the main method.
2. **static:** The static keyword means the method belongs to the class itself, not to instances (objects) of the class. This is why the JVM can call the main method without creating an object of the class.
3. **void:** This keyword means the main method doesn't return any value.
4. **main:** This is the name of the method that the JVM looks for to start the program.
5. **String[] args:** This is an array of String values that can take command-line arguments, allowing input when the program starts.

Real-World Analogy: Front Door

Think of the main method as the front door of a house. When you want to enter a house, you go through the front door. Similarly, when the JVM starts a Java program, it looks for the main method as the "front door" to enter and start executing code.

Example of Main Method Usage

```Java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

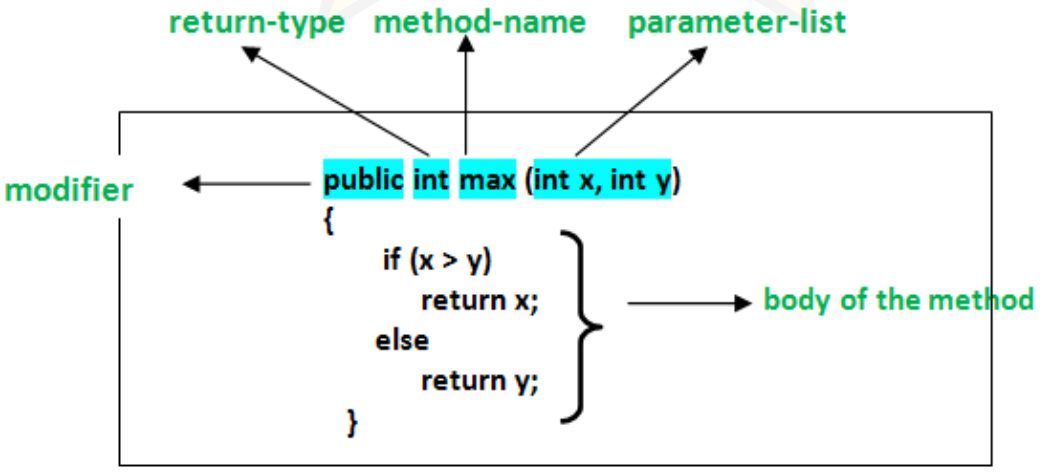
Explanation: When you run this code, the main method is called, and it prints "Hello, World!" to the screen.

Real-Time Usage of Main Method

In larger applications, the main method usually serves as the starting point where setup or initialization tasks are done. After that, it might call other methods to perform specific tasks.

Methods

Method: This is a block of code which is a behaviour of a class, and which performs a specific task. And whenever we call them they will perform the action / operation whatever we tell them to do



Types of Methods in Java

1. **Predefined Methods:** Methods that Java provides, like System.out.println() to print on the console.
2. **User-Defined Methods:** Methods created by the programmer to perform specific tasks in a program.

A real-world analogy for a class and its methods is a **"Television Remote"**:

1. **Class**: TelevisionRemote
   - This represents the "blueprint" for a TV remote with specific buttons and functions.
2. **Attributes** or Variables (Data Members):
   - batteryLevel
   - brand
   - model
3. **Methods** (Behaviors):
   - turnOn(): Turns the television on.
   - turnOff(): Turns the television off.
   - increaseVolume(): Raises the volume level.
   - decreaseVolume(): Lowers the volume level.
   - changeChannel(channelNumber): Switches the television to a specified channel.

In this analogy:

The **remote** is like a **class** in programming.

Each **button** on the remote represents a **method**.

When we press a button (call a method), the remote performs a specific action on the TV (object) based on its design.

Just as a class defines what methods an object will have, the remote provides a set of actions it can perform.

Method Syntax: Basic syntax of a method in Java:

```java
returnType methodName(parameters) {
    // method body
}
```

returnType: The data type of the value that the method will return. If the method doesn't return any value, the return type is void.

methodName: The name of the method.

parameters: Values that can be passed to the method to work with. These are optional.

Example of a Simple Method

Let's create a simple method that adds two numbers:

```java
public class MathOperations {
    public static int add(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        int result = add(5, 10);  // Calling the add method
        System.out.println("Sum: " + result);
    }
```

```
}

// Output will be Sum: 15
```

Explanation: Here, we created a method add that takes two numbers (a and b), adds them, and returns the result. The main method then calls add(5, 10) to get the sum.

Mechanisms Involved in Methods

1. **Method Calling:** When you want to use a method, you "call" it by using its name followed by parentheses. In the example above, add(5, 10) is a method call.
2. **Method Parameters and Arguments:** Parameters are like placeholders that let a method accept different values each time it's called. When calling a method, you pass actual values known as arguments.
3. **Return Type:** Methods can return a value, which is defined by the returnType. For example, if a method returns an integer, you use int as the return type.

Real-Time Usage of Methods

Methods are used extensively in real-world applications for various purposes, like calculations, data processing, and interaction with users. They make code modular, organized, and reusable.

Now let see the Types of methods in Java

Non-parameterized Methods

Parameters are the local variables of a method which are given in the method signature and a method with no parameters is simply known as **non-parameterized method**, and whenever we define a method what and what we give it, it is simply called a method **signature**

Example:

```java
class PrintHelloWorld {
    void print() {
        System.out.println("Hello World");
      }
      public static void main (String[] args) {
      PrintHelloWorld h = new PrintHelloWorld();

      h.print();
      h.print();
      }
}
```

OUTPUT: **HelloWorld**
HelloWorld

Parameterized Method

If we want a method to take input and perform operation on the input then we can define variables in the method body that will have parameters. And this method is simply called as Parameterized method

Example

```java
class PrintNameAge{
    void print(String name, int age) { // defining variables
     System.out.println(name +" "+ age);
}
      public static void main (String[] args) {
      // PrintNameAge Object Created
      PrintNameAge c = new PrintNameAge();
      // print() method called using referance variable c
      c.print("Vishwa",100);
      }
}
```

```java
Output : Vishwa 100
```

```java
public class MathOperations {
    public static int add(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        int result = add(5, 10);  // Calling the add method
        System.out.println("Sum: " + result);  // Output will be Sum: 15
    }
}
```

OUTPUT

```java
Sum: 15
```

Instance Methods:

These methods belong to an object and need an object to be called.

Example

```java
class Caliculator {
// Instance Method
    void add(int a, int b) {
        System.out.println(a + b);
      }
      public static void main (String[] args) {
            // Caliculator Object Created
            Caliculator c = new Caliculator();
            // add() method called using referance variable c
            c.add(10,20);
      }
}
```

OUTPUT**: 30**

Static Methods

These methods belong to the class itself and can be called without creating an object.

To make a method static we have use static keyword in method signature like below

Example

```
Java
class Caliculator {

    // Static Method
    static void substract(int a, int b) {
        System.out.println(a-b);
    }
      public static void main (String[] args) {
      substract(10,30);
      }
}
```

OUTPUT **-20**

Above method as called without creating the object of the Calculator class

Returning Methods

These are the methods which will return us a result after their execution, and we can achieve with **return** keyword and return keyword what it will return is completely dependent on the **return type** of a method

Example:

```
Java

class Caliculator {        // Instance Method
int add(int a, int b) {   // return type is int
return a + b;            // returning int type of data
}
public static void main (String[] args) {
Caliculator cal = new Caliculator();
      int res = cal.add(10,20);
      System.out.println(res);
   }
}
```

OUTPUT **30**

Above code consist of a method whose return type is 'int' and is returning a integer value

Not Returning Methods / Void Methods

If we want a method not return then we can use the key word called **void** in methods signature

Let's see the example of void or not returning method

Example

```java
class PrintNum { // Instance Method
    void add(int a) { // and return type is void
        System.out.println(a); // prints the result
    }

    public static void main(String[] args) {
        PrintNum print = new PrintNum();
        print.add(10);
    }
}
```

OUTPUT: **10**

THANK YOU

AIMERZ.ai

Aim.Act.Achieve